Deep Feedforward Networks

Lecture Notes Based on Chapter 6 of *Deep Learning*

Based on Ian Goodfellow, Yoshua Bengio, and Aaron Courville

Contents

1 Lecture Overview **2**

2 Why Do We Need Deep Learning? **2**

3 What Is a Feedforward Network? **3**

4 The Basic Computational Unit: A Neuron **3**
4.1 Understanding each term 4

5 The Critical Problem: XOR **4**

6 Can One Perceptron Learn XOR? **4**
6.1 Visual understanding of XOR 5
6.2 What does "not linearly separable" mean? 5
6.3 Why is XOR important? 6

7 Solution: Add a Hidden Layer **6**
7.1 Express XOR using AND, OR, and NOT 6
7.2 A numerical threshold-neuron construction 6
7.3 Representation learning 7

8 Why Nonlinear Activation Functions Are Necessary **7**

9 Common Activation Functions **7**

10 ReLU Activation Function **8**
10.1 ReLU examples 8
10.2 ReLU graph 8
10.3 Why is ReLU popular? 9
10.4 A useful note about the ReLU derivative 9

11 Quick Review and Classroom Questions **9**

12 Key Takeaways **9**

1 Lecture Overview

For a Master's class, this chapter is best taught as a gradual transition:

Problem → Perceptron → XOR limitation → Hidden layer → Nonlinearity → ReLU → Deep feedforward network → Forward propagation → Loss → Gradient descent and backpropagation

Target audience: Master's students.
Assumed background: Basic algebra and elementary machine-learning concepts; advanced mathematics is not required.

Learning objectives

By the end of the lecture, students should be able to:

- Explain what a feedforward neural network is.
- Understand why a single perceptron cannot learn XOR.
- Explain how a hidden layer can solve XOR.
- Explain why nonlinear activation functions are necessary.
- Calculate a neuron's output step by step.
- Define and calculate the ReLU activation.
- Describe the architecture of a deep feedforward network.
- Explain forward propagation using equations.
- Describe the roles of weights, biases, and activation functions.
- Solve simple numerical problems involving a neural network.

2 Why Do We Need Deep Learning?

Start with this question:

> Can a machine learn every pattern using a simple linear model?

Consider a student-pass prediction problem. Let

$$x_1 = \text{study hours}, \qquad x_2 = \text{attendance}.$$

The target is

$$y = \begin{cases} 1 & \text{if the student passes,} \\ 0 & \text{if the student fails.} \end{cases}$$

A simple linear model computes

$$z = w_1 x_1 + w_2 x_2 + b,$$

and then applies an activation function:

$$y = \text{activation}(z).$$

For example,

$$z = 2x_1 + 3x_2 - 5.$$

The model learns parameters that determine a decision boundary. A linear model can solve many tasks, but it cannot represent every possible input-output relationship.

3 What Is a Feedforward Network?

A feedforward network processes information from input to output. In a standard feedforward network, information does not travel around recurrent loops.

$$\boxed{\text{Input} \longrightarrow \text{Computation} \longrightarrow \text{Output}}$$

Figure 1 shows a simple network with an input layer, a hidden layer, and an output layer.

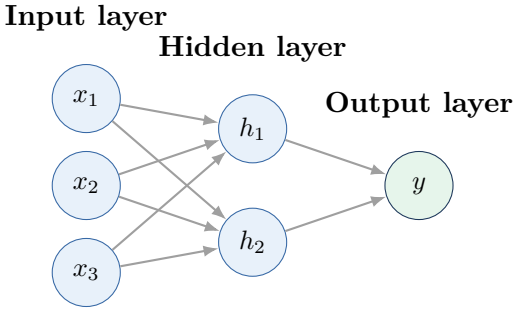


Figure 1: A simple feedforward neural network.

The network transforms

$$x \longrightarrow h \longrightarrow y,$$

where x is the input, h is a hidden representation, and y is the output.

4 The Basic Computational Unit: A Neuron

Before discussing deep networks, students should understand one neuron.

Suppose the inputs are

$$x_1 = 2, \qquad x_2 = 3,$$

the weights are

$$w_1 = 0.5, \qquad w_2 = 0.2,$$

and the bias is $b = 1$.

The neuron first calculates the weighted sum:

$$z = w_1 x_1 + w_2 x_2 + b.$$

Substituting the values,

$$\begin{aligned} z &= (0.5)(2) + (0.2)(3) + 1 \\ &= 1 + 0.6 + 1 \\ &= 2.6. \end{aligned}$$

Thus,

$$\boxed{z = 2.6}.$$

The neuron then applies an activation function:

$$a = f(z).$$

Its overall computation is

$$\boxed{a = f(w^\top x + b)}.$$

This is one of the central equations in the chapter.

4.1 Understanding each term

- x – **input/features:**
$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}.$$
- w – **weights:**
$$w = \begin{bmatrix} 0.5 \\ 0.2 \end{bmatrix}.$$
- $w^\top x$ – **weighted sum:**
$$w^\top x = (0.5)(2) + (0.2)(3) = 1.6.$$
- b – **bias:** a trainable offset that shifts the neuron's pre-activation value.
- f – **activation function:** a function applied to the weighted sum. A nonlinear activation allows the network to represent nonlinear relationships.

5 The Critical Problem: XOR

XOR means *exclusive OR*. Its output is 1 when the two binary inputs differ and 0 when they are the same.

Table 1: XOR truth table

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Equivalently,
$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

6 Can One Perceptron Learn XOR?

A single perceptron computes
$$z = w_1 x_1 + w_2 x_2 + b$$
and may use a threshold activation:
$$y = \begin{cases} 1 & z > 0, \\ 0 & z \le 0. \end{cases}$$

Its decision boundary is
$$w_1 x_1 + w_2 x_2 + b = 0.$$
For two input features, this boundary is a straight line.

6.1 Visual understanding of XOR

The four XOR input points are

$$(0,0) \mapsto 0, \quad (0,1) \mapsto 1, \quad (1,0) \mapsto 1, \quad (1,1) \mapsto 0.$$

The two positive points occupy opposite corners, as do the two negative points. No single straight line can separate both classes.

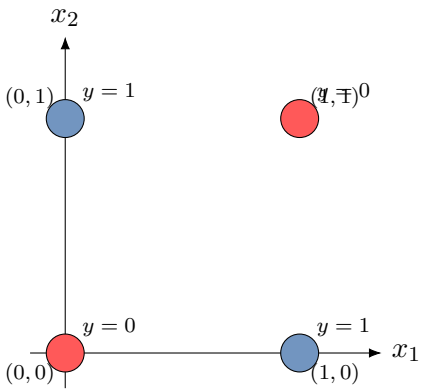


Figure 2: XOR data points. Positive and negative examples alternate across the corners.

Ask the class: Can you draw one straight line that separates both 1s from both 0s?

No

6.2 What does "not linearly separable" mean?

A dataset is linearly separable if a line, plane, or hyperplane can separate its classes.

AND

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

AND is linearly separable.

OR

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

OR is also linearly separable.

XOR is not linearly separable. Therefore,

A single perceptron cannot learn XOR.

6.3 Why is XOR important?

XOR is a small problem that exposes a fundamental limitation: a single linear neuron cannot represent every function. This motivates the use of multiple neurons, hidden layers, and nonlinear activation functions.

7 Solution: Add a Hidden Layer

Instead of connecting inputs directly to the output, add a hidden layer:

$$\text{Input} \longrightarrow \text{Hidden layer} \longrightarrow \text{Output}.$$

The hidden neurons learn intermediate representations. This is the key conceptual transition:

$$\boxed{x \longrightarrow h \longrightarrow y}.$$

7.1 Express XOR using AND, OR, and NOT

A useful logical identity is

$$\boxed{\text{XOR}(x_1, x_2) = (x_1 \,\text{OR}\, x_2) \,\text{AND}\, \text{NOT}(x_1 \,\text{AND}\, x_2)}.$$

Define hidden neurons as

$$h_1 = x_1 \,\text{OR}\, x_2, \qquad h_2 = x_1 \,\text{AND}\, x_2.$$

Then the output is

$$y = h_1 \,\text{AND}\, \text{NOT}(h_2).$$

The hidden layer transforms the original input into a representation that makes the final decision possible.

Table 2: XOR represented by hidden features

x_1	x_2	$h_1 = \text{OR}$	$h_2 = \text{AND}$	$y = h_1 \wedge \neg h_2$
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

7.2 A numerical threshold-neuron construction

The logic above can be implemented with threshold neurons. Let

$$\text{step}(z) = \begin{cases} 1 & z > 0, \\ 0 & z \leq 0. \end{cases}$$

The OR hidden neuron can be defined as

$$h_1 = \text{step}(x_1 + x_2 - 0.5),$$

and the AND hidden neuron as

$$h_2 = \text{step}(x_1 + x_2 - 1.5).$$

The output neuron is

$$y = \text{step}(h_1 - h_2 - 0.5).$$

Check all four cases:

- $(0,0)$: $h_1 = 0$, $h_2 = 0$, so $y = 0$.
- $(0,1)$: $h_1 = 1$, $h_2 = 0$, so $y = 1$.
- $(1,0)$: $h_1 = 1$, $h_2 = 0$, so $y = 1$.
- $(1,1)$: $h_1 = 1$, $h_2 = 1$, so $y = 0$.

Thus, the hidden layer enables a network of threshold neurons to represent XOR.

7.3 Representation learning

The original input representation is (x_1, x_2). The hidden representation is (h_1, h_2). The network learns useful intermediate features rather than relying only on the original input coordinates.

8 Why Nonlinear Activation Functions Are Necessary

Suppose every layer performs only an affine transformation:

$$h = W_1 x + b_1, \qquad y = W_2 h + b_2.$$

Substituting the first equation into the second gives

$$\begin{aligned} y &= W_2(W_1 x + b_1) + b_2 \\ &= W_2 W_1 x + W_2 b_1 + b_2. \end{aligned}$$

This is still an affine transformation of x. Therefore, stacking affine layers without nonlinear activations does not provide the expressive power expected from a deep network.

With a nonlinear activation function,

$$\boxed{h = f(Wx + b)},$$

the network can learn nonlinear relationships. In short,

$$\text{many affine layers without nonlinearities} \;\equiv\; \text{one affine transformation.}$$

9 Common Activation Functions

Common activation functions include:

- Step function
- Sigmoid
- Hyperbolic tangent (tanh)
- Rectified Linear Unit (ReLU)
- Leaky ReLU
- Softmax

For this lecture, the main focus is ReLU.

10 ReLU Activation Function

ReLU stands for *Rectified Linear Unit*. It is defined as

$$\boxed{\text{ReLU}(x) = \max(0, x)}$$

or, equivalently,

$$\text{ReLU}(x) = \begin{cases} 0 & x < 0, \\ x & x \geq 0. \end{cases}$$

In simple terms:

- Negative input: output 0.
- Positive input: keep the input unchanged.
- Zero input: output 0.

10.1 ReLU examples

$$\text{ReLU}(5) = 5$$

$$\text{ReLU}(-3) = 0$$

$$\text{ReLU}(0) = 0$$

$$\text{ReLU}(2.7) = 2.7$$

$$\text{ReLU}(-8.4) = 0$$

Question for students: What is $\text{ReLU}(-2.5)$?

$$\boxed{\text{ReLU}(-2.5) = 0}$$

10.2 ReLU graph

The function is flat at zero for negative inputs and follows the line $y = x$ for nonnegative inputs.

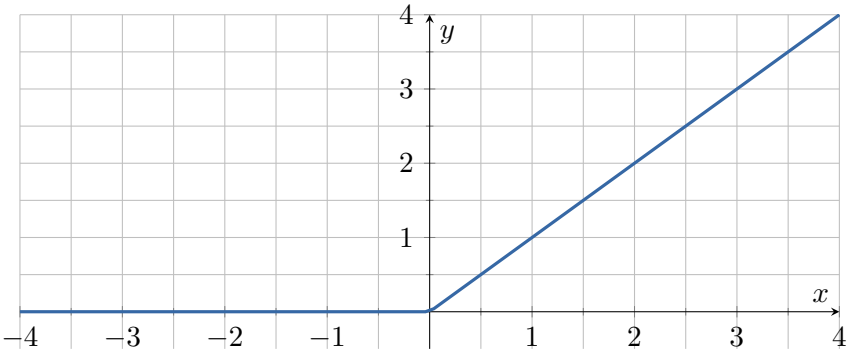


Figure 3: ReLU: $\text{ReLU}(x) = \max(0, x)$.

10.3 Why is ReLU popular?

1. **Simple:** $\mathrm{ReLU}(x) = \max(0, x)$.
2. **Computationally inexpensive:** it requires a comparison and selection rather than an exponential calculation.
3. **Nonlinear:** despite being piecewise linear, it allows a network to represent nonlinear functions when combined across layers.
4. **Often helps optimization:** on its positive side, its derivative is 1, so it avoids the positive-side saturation associated with sigmoid and tanh.

10.4 A useful note about the ReLU derivative

For $x < 0$, the derivative is 0. For $x > 0$, the derivative is 1. At $x = 0$, ReLU is not differentiable in the classical sense; implementations choose a subgradient convention, commonly 0.

$$\mathrm{ReLU}'(x) = \begin{cases} 0 & x < 0, \\ 1 & x > 0. \end{cases}$$

A limitation is that a neuron that remains in the negative region for all its inputs may receive zero gradient through ReLU and stop updating through that path. This is often called the *dying ReLU* problem.

11 Quick Review and Classroom Questions

1. What is a feedforward neural network?
2. What do weights and biases do in a neuron?
3. Why can a single perceptron represent AND and OR but not XOR?
4. What does it mean for a dataset to be linearly separable?
5. How does a hidden layer help solve XOR?
6. Why are nonlinear activation functions necessary?
7. Calculate $\mathrm{ReLU}(-4)$, $\mathrm{ReLU}(0)$, and $\mathrm{ReLU}(3.5)$.
8. What can happen if a network stacks many affine layers without nonlinear activation functions?

12 Key Takeaways

- A neuron computes a weighted sum plus a bias and applies an activation function.
- A single perceptron has a linear decision boundary and cannot represent XOR.
- A hidden layer can construct intermediate features that make XOR solvable.
- Stacking affine layers alone does not create a nonlinear model.
- Nonlinear activations are essential for the expressive power of deep feedforward networks.
- ReLU is simple and computationally efficient, and often supports effective gradient-based training.

Reference

Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning.* MIT Press, 2016. Chapter 6, "Deep Feedforward Networks." Available at: https://www.deeplearningbook.org/contents/mlp.html